NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC2002: Object-Oriented Design & Programming

Topic 10: Modern Java Enhancements

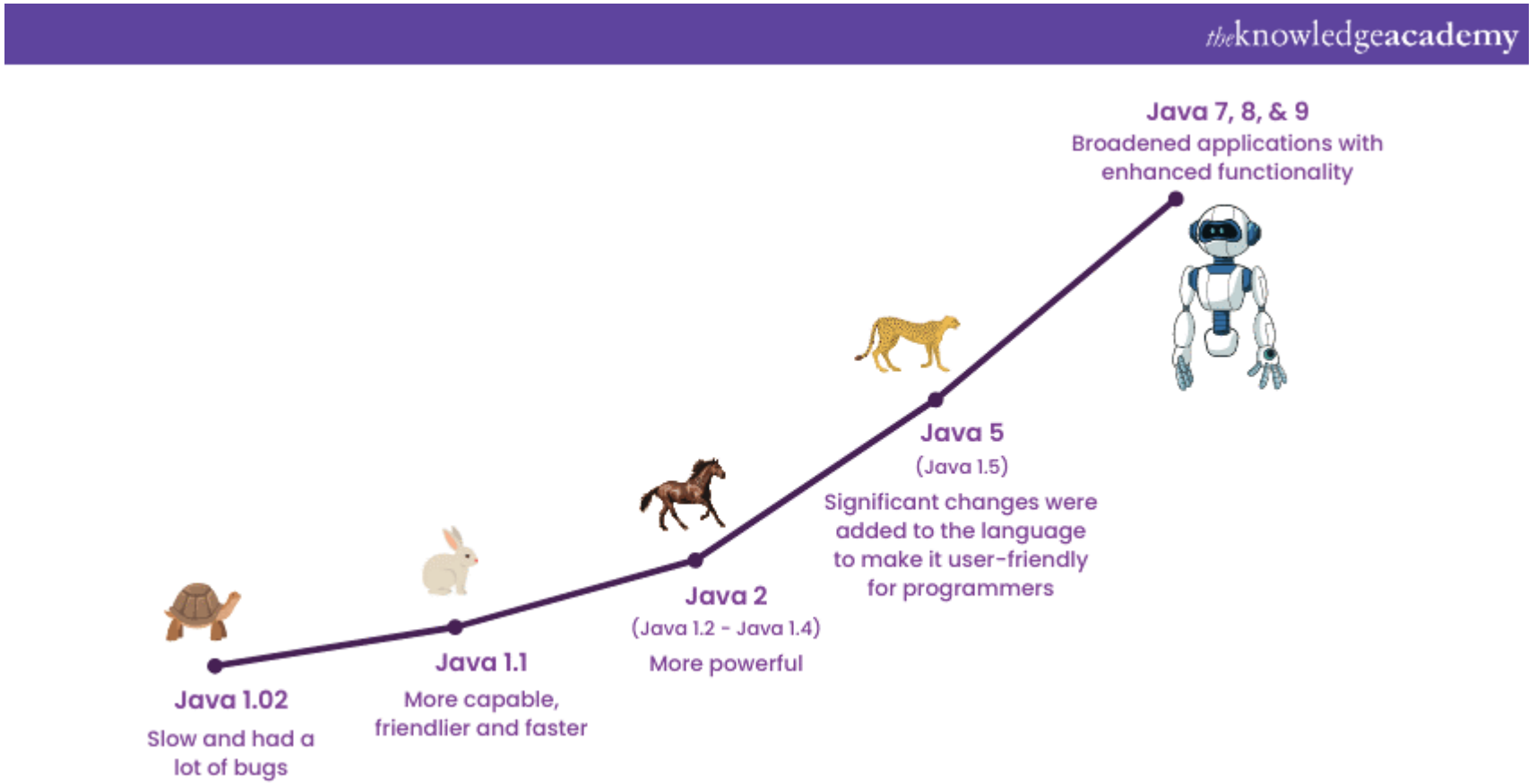
Dr Li Fang, College of Computing and Data Science (CCDS)

Objective

- Understand the fundamentals and benefits of Generics introduced in Java 5.
- Explore enhancements in Collections and the Enhanced For Loop (Java 5+).
- Gain hands-on knowledge of Lambda Expressions and the Streams API, key functional programming features in Java 8.
- Learn how Default Methods in Interfaces improve code reusability and API evolution.
- Examine the Enhanced Switch statement introduced in Java 12+ for cleaner and more readable code.

The Evolution of Java

- Java 1.x → Java 5 → Java 8 → Java 12+



Topic 10.1: INTRODUCTION TO GENERICS (JAVA 5)

Code Before Java 5 (Without Generics)

```
import java.util.ArrayList;
import java.util.List;


public class Test {
public static void main(String[] args) {
List myList = new ArrayList(); // Raw type (no type safety)
// Adding elements (Objects of different types)
```

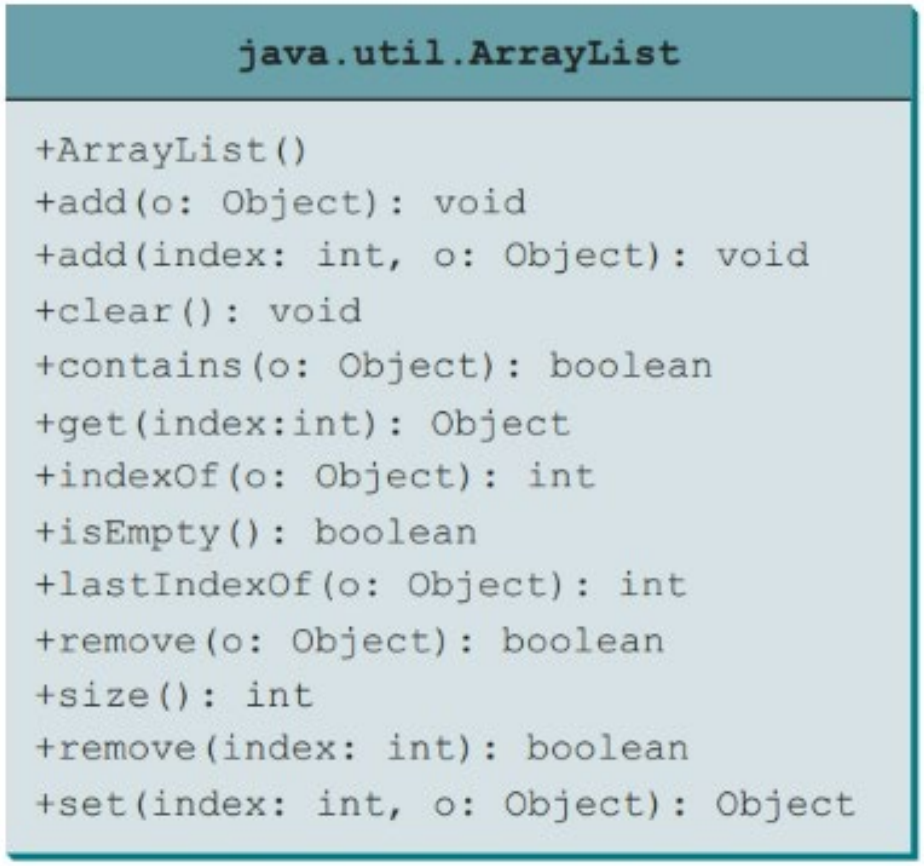
```
Java
Exception in thread "main" java.lang.ClassCastException: class java.lang.Integer cannot be cast to class java.lang.String
```

```
// Retrieving elements (Requires explicit casting)
String str = (String) myList.get(0); // Works fine
System.out.println(str);
String num = (String) myList.get(1); // Compiles but throws ClassCastException at
runtime
System.out.println(num); // Runtime error occurs here
}
}
```

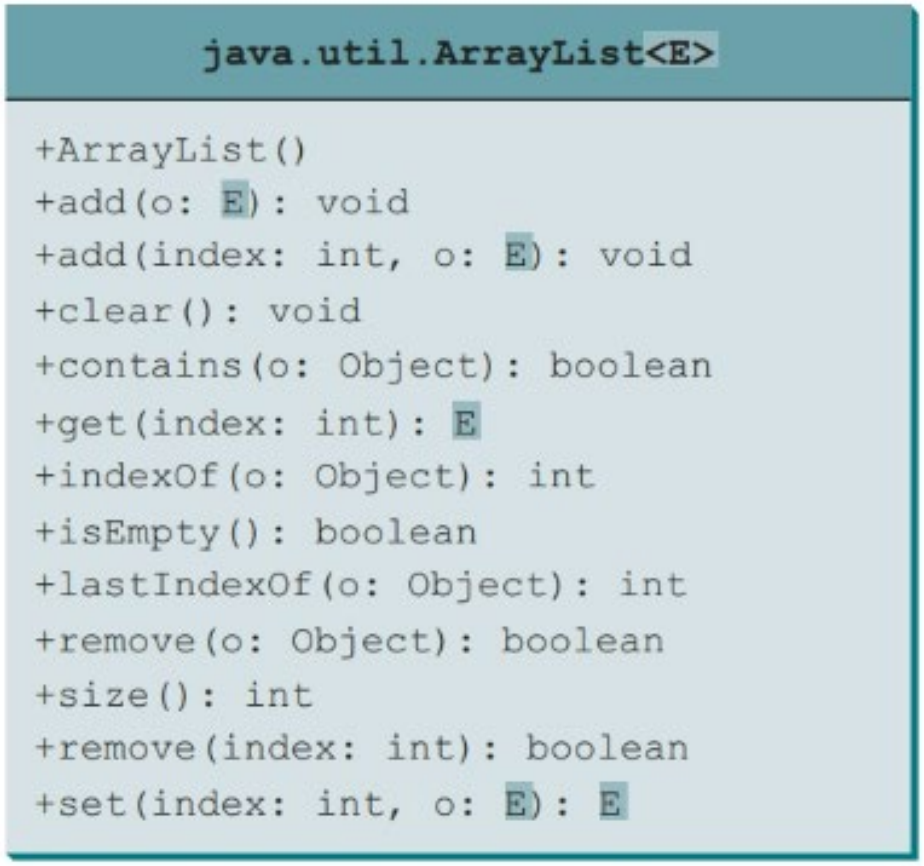
Solution with Generics (Java 5+)

```
import java.util.ArrayList;
import java.util.List;


public class GenericExample {
public static void main(String[] args) {
List<String> myList = new ArrayList<>(); // Type safety enforced
// Adding elements (Objects of different types)
myList.add("Java");
myList.add(2); // Compile-time error: prevents incorrect type


String str = myList.get(0); // No casting needed
System.out.println(str);
String num = (String) myList.get(1); // Compiles but throws ClassCastException at runtime
System.out.println(num); // Runtime error occurs here
}
}
```

ArrayList ClassDiagram comparison



(a) `ArrayList` before JDK 1.5



(b) `ArrayList` since JDK 1.5

The generic class definition of ArrayList in Java

```
public class ArrayList<E> implements Iterable<E> {
private static final int DEFAULT_CAPACITY = 10;
private Object[] elements;
private int size = 0;


// Constructors
public ArrayList() {
elements = new Object[DEFAULT_CAPACITY];
}

....
// Add an element
public boolean add(E element) {
ensureCapacity(size + 1);
elements[size++] = element;
return true;
}
```

Improvement: The Diamond Operator in Java 7

•**Example: Java 5 Generics-** required explicit type declaration on both sides.

```
List<String> list = new ArrayList<String>(); // Before Java 7
```

•**Example: Java 7 Diamond Operator-**allows type inference on the right-hand side, reducing redundancy.

```
List<String> list = new ArrayList<>(); // Java 7+
```

Custom Generic Class – Box<T>

```
public class Box<T> {

    private T content;

    public void set(T content) { this.content = content; }

    public T get() { return content; }

}
```

```
Box<Integer> intBox = new Box<>();
intBox.set(123);
Integer number = intBox.get(); // Returns Integer
```

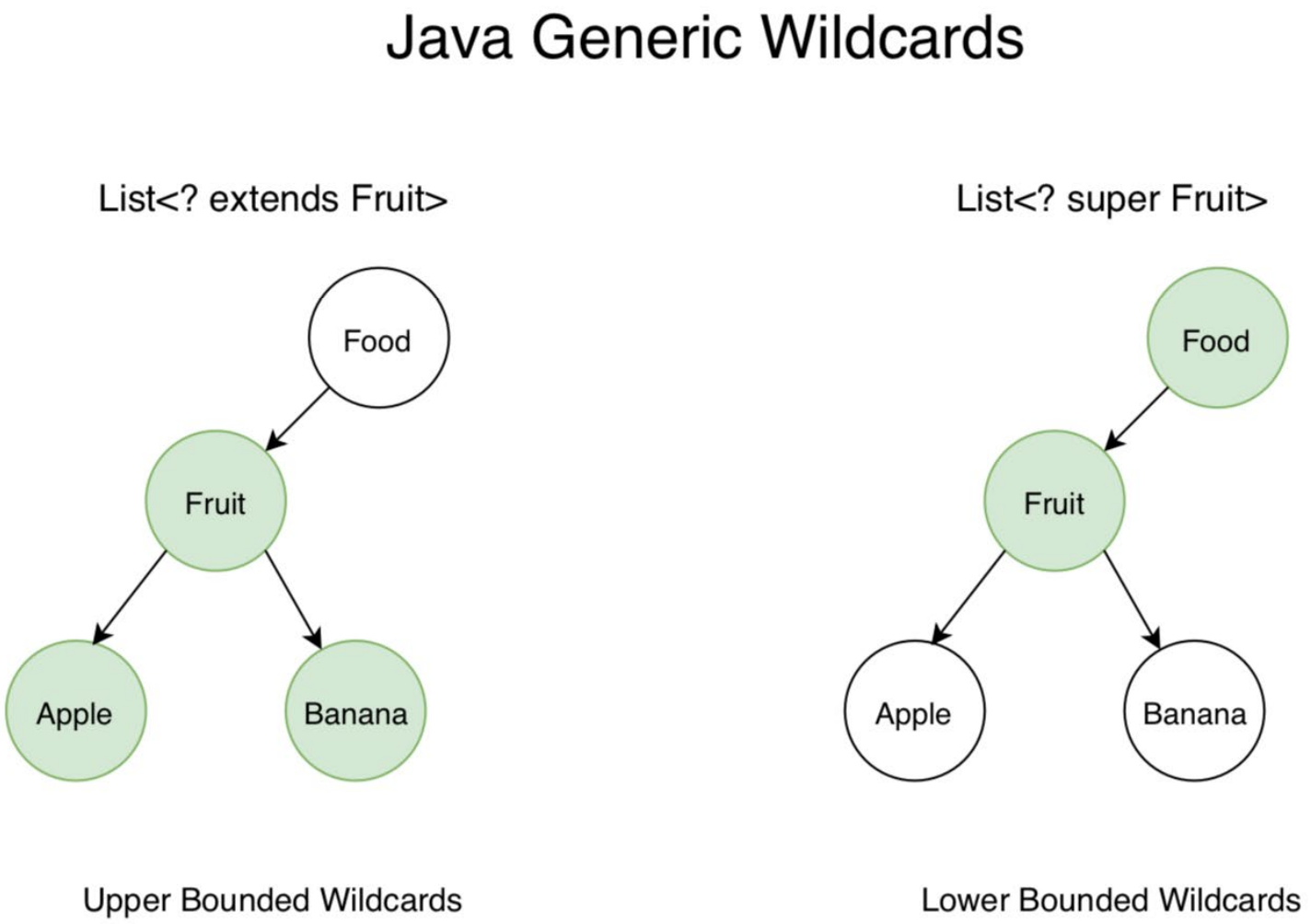
Multiple Type Parameters

```
//Generic Box class with two type parameters: T and U
class Box<T, U> {
private T first;
private U second;


// Constructor
public Box(T first, U second) {
this.first = first;
this.second = second;
}


// Getters
public T getFirst() {
return first;
}


public U getSecond() {
return second;
}…..
```

```
Box<String, Double> priceBox = new Box<>("Laptop", 799.99);
System.out.println("Item: " + priceBox.getFirst() + ", Price: $" + priceBox.getSecond());
```

Item: Laptop, Price: $799.99

Generics – Bounded Wildcards



Generics – Unbounded Wildcards

An unbounded wildcard (<?>) is used when we don’t know or don’t care about the specific type, but we still want to work with a generic class.

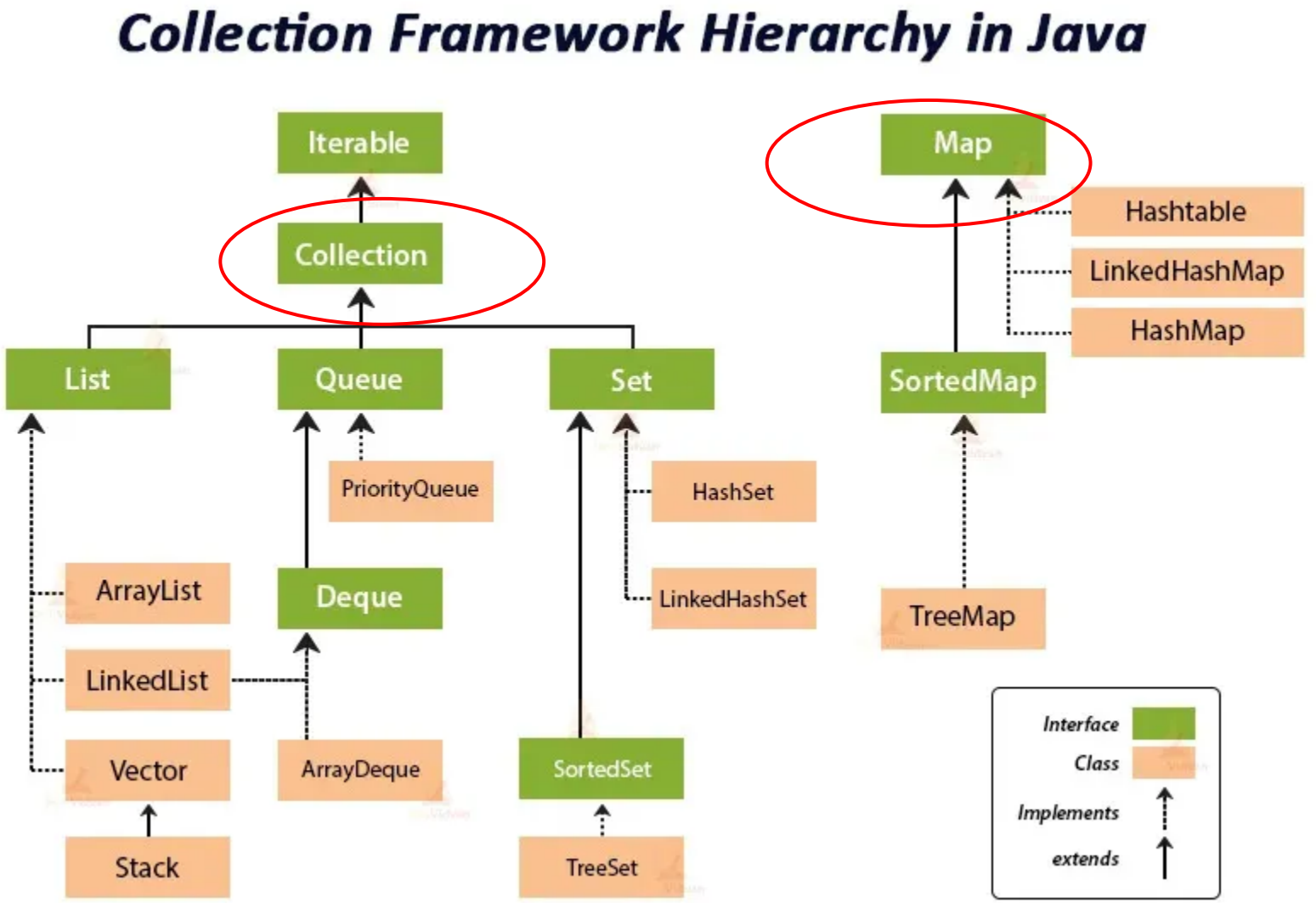
```
public class UnboundedWildcardExample {
// Method using an unbounded wildcard
public static void printList(List<?> list) {
for (Object element : list) {
System.out.println(element);
}
}


public static void main(String[] args) {
List<String> stringList = Arrays.asList("Apple", "Banana", "Cherry");
List<Integer> intList = Arrays.asList(10, 20, 30);


System.out.println("Printing String List:");
printList(stringList); // Works with List<String>


System.out.println("Printing Integer List:");
printList(intList); // Works with List<Integer>
}
}
```

Type Parameter Naming Conventions

E — Represents an **Element**, primarily used in the Java Collections framework (e.g., List<E>).
K — Represents a **Key**, commonly used for the key type in maps (e.g., Map<K, V>).
V — Represents a **Value**, typically used for the value type in maps (e.g., Map<K, V>).
N — Represents a **Number**, often used when dealing with numeric values.
T — Represents a **Type**, generally used for the first generic type parameter (e.g., Class<T>).
S — Represents the **Second Type** parameter in scenarios requiring multiple type parameters.
U — Represents the **Third Type** parameter in scenarios requiring multiple type parameters.
V — Represents the **Fourth Type** parameter in scenarios requiring multiple type parameters.

Topic 10.2: COLLECTIONS ENHANCEMENTS (JAVA 5+)

Java Collections Framework Hierarchy



Topic 10.3: ENHANCED FOR LOOP (JAVA 5)

Ways to Traverse a Collection in Java

- **Basic for loop** – Good for index-based access.
- **Enhanced for-each loop** – Simple, but no modifications allowed.
- **Iterator (Iterator<T>)** – Flexible and allows safe removal.
- **Stream API (forEach())** – Functional approach, useful for large data sets.

Basic for Loop (Index-Based Iteration)

```
List<String> list = Arrays.asList("Apple", "Banana", "Cherry");
for (int i = 0; i < list.size(); i++) {
System.out.println(list.get(i)); // Access elements using index
}
```

Enhanced for-each Loop (Simplified Iteration)

Syntax of Enhanced For Loop:
for (Type variable : collection) { // Code to be executed }
✓ **Type** – The data type of elements in the collection/array.
✓ **variable** – A temporary variable that holds the current element during iteration.
✓ **collection** – The array or collection being iterated over.

```
List<String> list = Arrays.asList("Apple",
"Banana", "Cherry");
for (String fruit : list) {
System.out.println(fruit);
}
```

Using an Iterator (Safe Element Removal)

```
Iterator<String> iterator = list.iterator();
while (iterator.hasNext()) {
    String fruit = iterator.next();
    System.out.println(fruit);
    if (fruit.equals("Banana")) {
        iterator.remove();  // Safe removal
    }
}
```

✓ Key Rule: Always use iterator.remove() to delete elements during iteration.
✓ Avoid list.remove() inside the loop, as it causes ConcurrentModificationException.

Java 8 Stream API + forEach() (Functional Approach)

- list.stream().forEach(item -> System.out.println("Element: " + item));

•.stream() creates a **stream** from the list.
•.forEach() iterates over each element.
•The **lambda expression** item -> System.out.println("Element: " + item) prints each item.

Topic 10.4: LAMBDA EXPRESSIONS (JAVA 8)

Traditional Method (Using a Class with a Method)

```
//Traditional method using a class and method
public class HelloWorld {
public void sayHello() {
System.out.println("Hello, World!");
}


public static void main(String[] args) {
HelloWorld obj = new HelloWorld();
obj.sayHello();
}
}
```

Using an Anonymous Class

```
interface Greeting {
void sayHello();
}


public class Main {
public static void main(String[] args) {
Greeting greeting = new Greeting() {
public void sayHello() {
System.out.println("Hello, World!");
}
};
greeting.sayHello();
}
}
```

Using a Lambda Expression (Simplified Code)

```
interface Greeting {
void sayHello();
}


public class Main {
public static void main(String[] args) {
Greeting greeting = () -> System.out.println("Hello, World!");
greeting.sayHello();
}
}
```

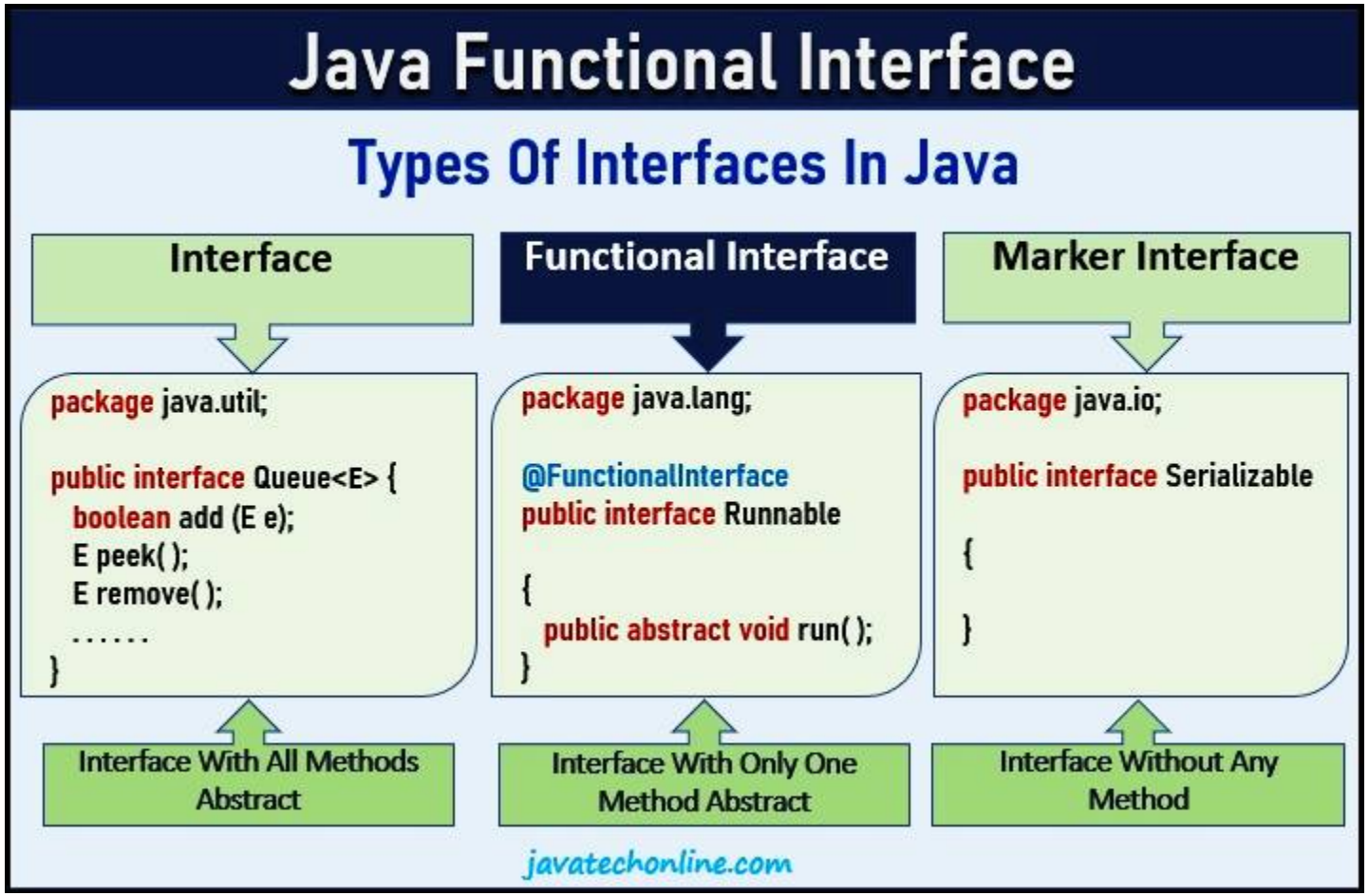
Lambda Expressions – Basic Syntax

(arguments) -> { body }

```
() -> System.out.println("Hello, World!");
```

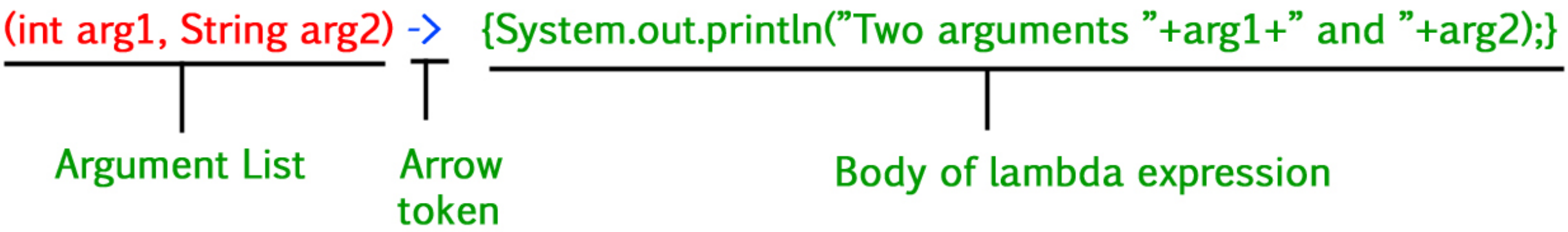
```
interface Greeting {
void sayHello();}


public class Main {
public static void main(String[] args) {
Greeting greeting = () -> System.out.println("Hello, World!");
greeting.sayHello();
}}
```

Understanding Functional Interfaces for Lambda Expressions



Lambda Expressions – Basic Syntax

(arguments) -> { body }



A simple lambda expression usage example

```
import java.util.function.BiCons
public class LambdaExample {
public static void main(String[]
BiConsumer<Integer, String> myLa
System.out.println("Two argument
};
```

```
@FunctionalInterface
public interface BiConsumer<T, U> {

    /**
     * Performs this operation on the given arguments.
     *
     * @param t the first input argument
     * @param u the second input argument
     */
    void accept(T t, U u);
```

```
myLambda.accept(5, "Hello"); // Output: Two arguments: 5 and
Hello
}
}
```

Lambda Expressions – Basic Syntax

Lambda Syntax

- No arguments:
```
() -> System.out.println("Hello")
```
- One argument:
```
s -> System.out.println(s)
```
- Two arguments:
```
(x, y) -> x + y
```
- With explicit argument types:
```
(Integer x, Integer y) -> x + y
```
- Multiple statements:
```
(x, y) -> {
    System.out.println(x);
    System.out.println(y);
    return (x + y);
}
```

Why Use Lambda Expressions?

✓ **Shorter Code** → Removes unnecessary method names and return keywords.
✓ **Easier to Read** → Less cluttered and more understandable.
✓ **Better for Functional Interfaces** → Works well with Java's Consumer, BiFunction, etc.

Method References

Know all five kinds of method references

All are at times preferable to lambdas

Type	Example	Lambda Equivalent*
Static	`Integer::parseInt`	`str -> Integer.parseInt(str)`
Bound	`Instant.now()::isAfter`	`Instant then = Instant.now();` `t -> then.isAfter(t)`
Unbound	`String::toLowerCase`	`str -> str.toLowerCase()`
Class Constructor	`TreeMap<K,V>::new`	`() -> new TreeMap<K,V>()`
Array Constructor	`int[]::new`	`len -> new int[len]`

Lambda Expressions – Best Practices

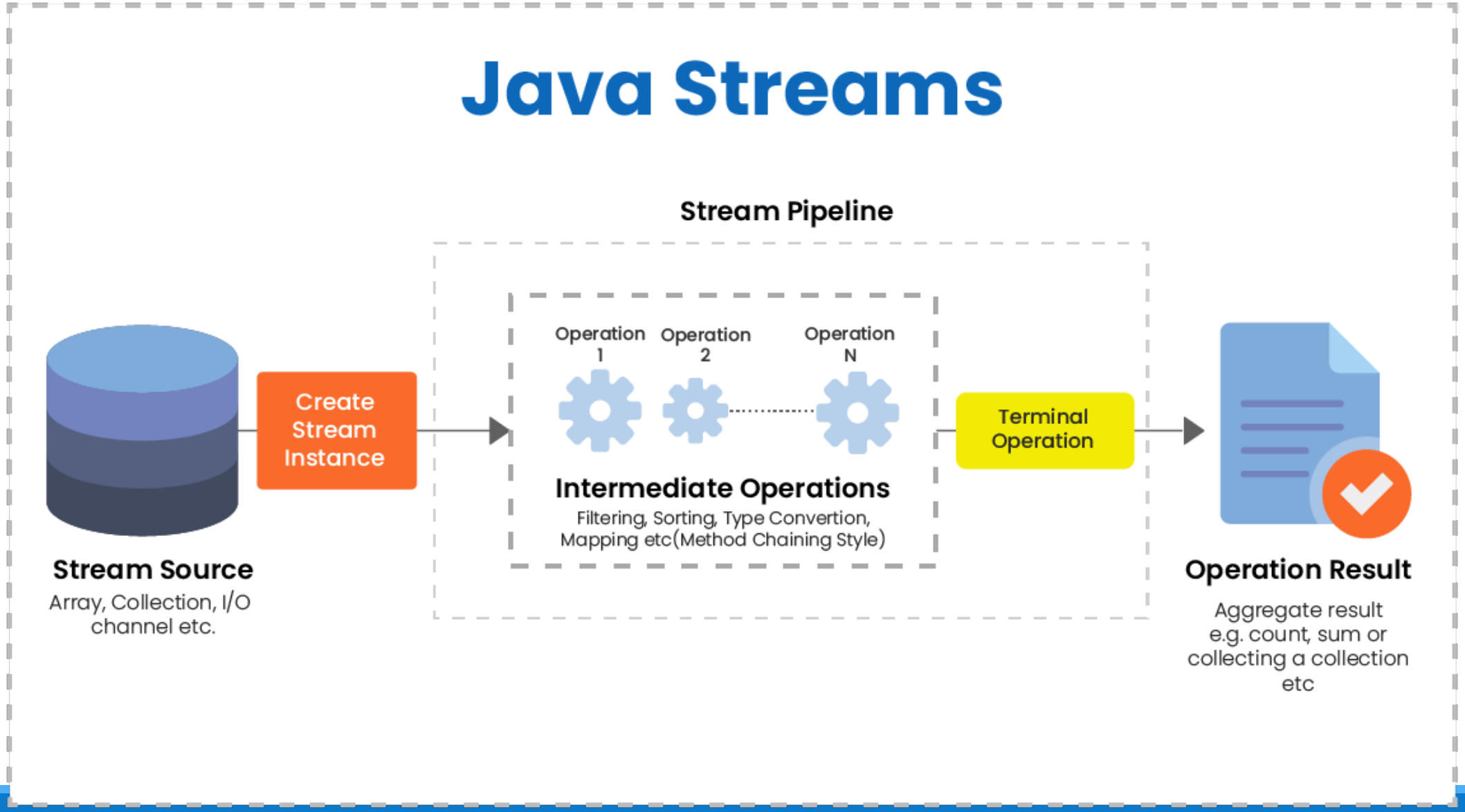
- ✅ **Keep lambdas short and focused**
  ✅ **Use method references where possible**
  ✅ **Avoid heavy logic within lambda bodies**

Topic 10.5: STREAMS API (JAVA 8)

Stream API example

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David");


List<String> filteredNames = names.stream()
.filter(name -> name.startsWith("A")) // Filter names starting with "A"
.map(String::toUpperCase) // Convert to uppercase
.collect(Collectors.toList()); // Collect results into a list


System.out.println(filteredNames); // Output: [ALICE]
```

What is Stream API?



Key Features of Java Streams

Feature	Description
Streams Do Not Store Data	They process data from existing sources like Collections and Arrays.
Immutable Processing	Streams do not modify the original data; they create new results.
Functional Programming Support	Allows method chaining and functional-style operations like `map()`, `filter()`, and `reduce()`.
Lazy Evaluation	Operations execute **only when needed**, improving efficiency.
Method Chaining	Multiple operations can be combined for **concise and readable** code.
Intermediate & Terminal Operations	**Intermediate:** `filter()`, `map()` (return streams). **Terminal:** `collect()`, `forEach()` (produce results).
Parallel Processing	Use `.parallelStream()` for **faster execution** on large datasets.

Streams – Filter Example

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");

List<String> filtered = names.stream()

                .filter(name -> name.startsWith("A"))

                .collect(Collectors.toList());
```

Filters elements starting with “A”

Streams – Map Example

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
List<Integer> squared = numbers.stream()
                .map(n -> n * n)
                .collect(Collectors.toList());
```

Transforms each element in the stream

Streams – Sorted Example

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
List<String> sortedNames = names.stream()
                .sorted()
                .collect(Collectors.toList());
```

Sorts elements in natural order

Streams – Terminal Operation: forEach

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
names.stream().forEach(System.out::println);
```

Consumes the stream, printing each element

Streams – Terminal Operation: Collect

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
List<String> result = names.stream()
                .filter(name -> name.length() > 3)
                .collect(Collectors.toList());
```

Aggregates elements into a collection

Streams – Parallel Streams

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
List<Integer> parallelResult = numbers.parallelStream()
                                      .map(n -> n * n)
                                      .collect(Collectors.toList());
```

Easy parallelization for performance gains

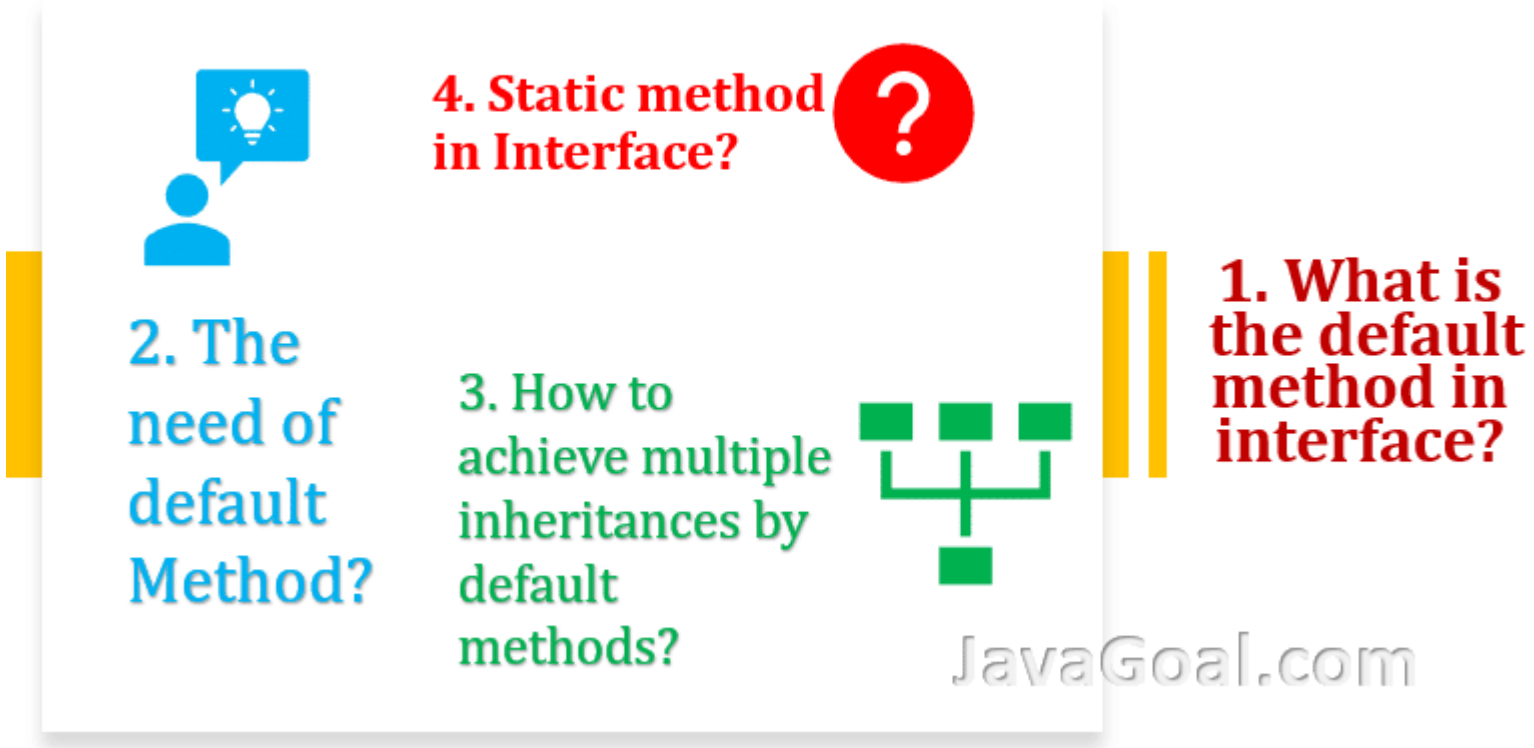
Complex Stream Example

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
List<String> finalResult = names.stream()
                .filter(n -> n.startsWith("A"))
                .map(String::toUpperCase)
                .sorted()
                .distinct()
                .collect(Collectors.toList());
```

Chained operations for complex transformations

Topic 10.6: DEFAULT METHODS IN INTERFACES (JAVA 8)

Understanding Default Methods in Java Interfaces



What is a Default Method in an Interface?

✅ Definition:
A default method in an interface is a method with a body (implementation), defined using the default keyword.
✅ Example:

```
interface Vehicle {
void start(); // Abstract method (must be implemented)


// Default Method with implementation
default void stop() {
System.out.println("Vehicle is stopping.");
}
}
```

Why Do We Need Default Methods?

Problem Before Default Methods (Pre-Java 8)

```
interface Printer {
void print();
}
```

✅ **Solution With Default Methods (Java 8+)**

```
interface Printer {
void print();


default void scan() {
System.out.println("Scanning document...");
}
}
```

How to Achieve Multiple Inheritance with Default Methods?

```
interface A {
default void show() {
System.out.println("Default method from A");
}
}


interface B {
default void show() {
System.out.println("Default method from B");
}
}


class C implements A, B {
@Override
public void show() {
System.out.println("Resolving conflict by overriding");
A.super.show(); // Choose method from interface A
}
}
```

What are Static Methods in Interfaces?

```
interface Utility {
static void displayMessage() {
System.out.println("This is a static method
in an interface.");
}
}
```

Summary & Takeaways

Feature	Purpose	Can Be Overridden?
Default Method	Allows adding new methods to interfaces without breaking existing code	✅ Yes
Static Method	Defines utility/helper functions inside an interface	❌ No
Abstract Method	Must be implemented by classes	✅ Yes

Topic 10.7: ENHANCED SWITCH (JAVA 12+)

Traditional Switch vs. Enhanced Switch

Traditional Switch Example:

```
int num;
switch (day) {
case MONDAY:
num = 1;
break;
...
default:
num = 0;
}
```

Enhanced Switch Example:

```
int num = switch (day) {
case MONDAY -> 1;
...
default -> 0;
};
```

Using Arrow Labels

```
int result = switch (grade) {
case 'A', 'B' -> 100;
case 'C' -> 75;
default -> 50;
};
```

Multiple labels in a single case

Using yield in Switch Expressions

```
int finalScore = switch (grade)
{
case 'A' -> {
System.out.println("Excellent");
yield 100;
}
case 'B' -> 85;
default -> 50;
};
```

yield returns a value from a block

Enhanced Switch – Enum Example

```
enum Day { MONDAY, TUESDAY, WEDNESDAY,
THURSDAY, FRIDAY, SATURDAY, SUNDAY }


String activity = switch(day) {
case SATURDAY, SUNDAY -> "Relax";
default -> "Work";
};
```

Enhanced switch pairs nicely with enums

Enhanced Switch – Benefits Recap

- **No Fall-Through:** Eliminates common errors
- **Expression Support:** Assign result directly to a variable
- **Concise and Cleaner Code**

Summary

Summary of Key Features

Java 5: Generics, Collections Enhancements, Enhanced For Loop

Java 8: Lambda Expressions, Streams API, Default Methods

Java 12+: Enhanced Switch

Major Benefits:

Type safety, functional programming, concise syntax, fewer runtime errors